



C++ - Módulo 03

Herencia

Resumen:

Este documento contiene los ejercicios del Módulo 03 de los módulos de C++.

Versión: 8.1

Contenido

I	Introducción	2
II	Reglas generales	3
III	Instrucciones de IA	6
IV	Ejercicio 00: Yyyy... ¡ABIERTO!	8
V	Ejercicio 01: Serena, mi amor!	10
VI	Ejercicio 02: Trabajo repetitivo	11
VII	Ejercicio 03: ¡Ahora es raro!	12
VIII	Presentación y evaluación por pares	14

Capítulo I

Introducción

C++ es un lenguaje de programación de propósito general creado por Bjarne Stroustrup como una extensión del lenguaje de programación C, o "C con clases" (fuente: [Wikipedia](#)).

El objetivo de estos módulos es introducirte en la Programación Orientada a Objetos. Este será el punto de partida de tu experiencia con C++. Muchos lenguajes son recomendables para aprender programación orientada a objetos, pero decidimos elegir C++ porque deriva de tu viejo amigo C. Debido a que se trata de un lenguaje complejo y para mantener las cosas simples, su código cumplirá con el estándar C++98.

Sabemos que el C++ moderno es muy diferente en muchos aspectos. Por lo tanto, si quieres convertirte en un desarrollador competente de C++, ¡de ti depende ir más allá de los 42 estándares Common Core!

Capítulo II

Reglas generales

Compilando

- Compila tu código con `c++` y los indicadores `-Wall -Wextra -Werror`
- Su código aún debería compilarse si agrega el indicador `-std=c++98`

Convenciones de formato y nomenclatura

- Los directorios de ejercicios se nombrarán de esta manera: `ex00`, `ex01`, ... , `exn`
- Nombra tus archivos, clases, funciones, funciones miembro y atributos según sea necesario en las directrices.
- Escriba los nombres de clase en formato Mayúsculas y minúsculas . Los archivos que contienen código de clase siempre se nombrarán según el nombre de la clase.
Por ejemplo: `ClassName.hpp/ClassName.h`, `ClassName.cpp` o `ClassName.hpp`. Si tiene un archivo de encabezado que contiene la definición de la clase "BrickWall", que representa un muro de ladrillos, su nombre será `BrickWall.hpp`.
- A menos que se especifique lo contrario, cada mensaje de salida debe terminar con un carácter de nueva línea y mostrarse en la salida estándar.

¡ Adiós, Norminette! No se impone ningún estilo de programación en los módulos de C++. Puedes seguir tu favorito. Pero recuerda que el código que tus pares evaluadores no pueden entender es código que no pueden calificar. Haz todo lo posible por escribir código limpio y legible.

Permitido/Prohibido

Ya no estás programando en C. ¡Hora de usar C++! Por lo tanto:

- Puedes usar casi todo lo de la biblioteca estándar. Por lo tanto, en lugar de limitarte a lo que ya sabes, sería inteligente usar las versiones C++ de las funciones de C que conoces siempre que sea posible.

Sin embargo, no puedes usar ninguna otra biblioteca externa. Esto significa que C++11 (y sus derivados) y las bibliotecas de Boost están prohibidas. Las siguientes funciones también están prohibidas: `*printf()`, `*alloc()` y `free()`. Si las usas, tu calificación será 0 y punto.

- Tenga en cuenta que, a menos que se indique explícitamente lo contrario, el uso del espacio de nombres `<ns_name>` y Las palabras clave de amistad están prohibidas. De lo contrario, tu calificación será -42.
- Solo se permite usar el STL en los módulos 08 y 09. Esto significa: no se permiten contenedores (vectoriales, de lista, de mapa, etc.) ni algoritmos (cualquier cosa que requiera incluir el encabezado `<algorithm>`) hasta entonces. De lo contrario, su calificación será de -42.

Algunos requisitos de diseño

- La pérdida de memoria también ocurre en C++. Cuando se asigna memoria (mediante el nuevo palabra clave), debe evitar fugas de memoria.
- Desde el Módulo 02 hasta el Módulo 09, sus clases deben estar diseñadas en el modelo Ortodoxo. Forma canónica, salvo que se indique explícitamente lo contrario.
- Cualquier implementación de función colocada en un archivo de encabezado (excepto las plantillas de función) significa 0 para el ejercicio.

Debes poder usar cada encabezado independientemente. Por lo tanto, deben incluir todas las dependencias necesarias. Sin embargo, debes evitar el problema de la doble inclusión añadiendo protecciones de inclusión. De lo contrario, tu calificación será 0.

Léame

- Puedes agregar archivos adicionales si lo necesitas (por ejemplo, para dividir tu código). Dado que estas tareas no están verificadas por un programa, puedes hacerlo siempre y cuando entregues los archivos obligatorios.
- A veces, las pautas de un ejercicio parecen cortas pero los ejemplos pueden mostrar requisitos que no están escritos explícitamente en las instrucciones.
- ¡Lee cada módulo completo antes de empezar! ¡De verdad, hazlo!

¡Por Odín, por Thor! ¡Usa tu cerebro!



Respecto al Makefile para proyectos C++, se aplican las mismas reglas que en C (ver el capítulo de Normas sobre el Makefile).



Tendrás que implementar muchas clases. Esto puede parecer tedioso, a menos que puedas programar tu editor de texto favorito.



Se le da una cierta cantidad de libertad para completar los ejercicios.
Sin embargo, sigue las reglas obligatorias y no seas perezoso.
¡Te pierdes mucha información útil! No dudes en leer sobre...
conceptos teóricos.

Capítulo III

Instrucciones de IA

- Context

Este proyecto está diseñado para ayudarle a descubrir los componentes fundamentales de su entrenamiento 42.

Para anclar adecuadamente los conocimientos y habilidades clave, es esencial adoptar un enfoque reflexivo al utilizar las herramientas y el soporte de IA.

El verdadero aprendizaje fundamental requiere un esfuerzo intelectual genuino, a través del desafío, la repetición y los intercambios de aprendizaje entre pares.

Para obtener una descripción más completa de nuestra postura sobre la IA (como herramienta de aprendizaje, como parte de la capacitación 42 y como una expectativa en el mercado laboral), consulte las preguntas frecuentes específicas en la intranet.

- Mensaje principal

Construye bases sólidas sin atajos.

Desarrollar realmente habilidades técnicas y de potencia.

Experimente el verdadero aprendizaje entre pares, comience a aprender a aprender y a resolver nuevos problemas.

El recorrido de aprendizaje es más importante que el resultado.

Conozca los riesgos asociados con la IA y desarrolle prácticas de control efectivas y contramedidas para evitar errores comunes.

- Reglas del alumno:

- Debes aplicar el razonamiento a las tareas asignadas, especialmente antes de recurrir a la IA.

- No debes pedir respuestas directas a la IA.
- Debes conocer los 42 enfoques globales sobre IA.

• Resultados de la fase:

Dentro de esta fase fundamental, obtendrás los siguientes resultados:

- Obtenga bases técnicas y de codificación adecuadas.
- Conozca por qué y cómo la IA puede ser peligrosa durante esta fase.

• Comentarios y ejemplo:

Sí, sabemos que la IA existe, y sí, puede resolver tus proyectos. Pero estás aquí para aprender, no para demostrar que la IA ha aprendido. No pierdas tu tiempo (ni el nuestro) solo para demostrar que la IA puede resolver el problema.

Aprender a los 42 años no se trata de saber la respuesta, sino de desarrollar la capacidad de encontrarla. La IA te da la respuesta directamente, pero eso te impide desarrollar tu propio razonamiento. Y razonar requiere tiempo, esfuerzo e implica fracaso. El camino al éxito no se supone que sea fácil.

- Tenga en cuenta que durante los exámenes, la IA no está disponible (no hay Internet, ni teléfonos inteligentes, etc.). Rápidamente se dará cuenta si ha confiado demasiado en la IA en su proceso de aprendizaje.

El aprendizaje entre pares te expone a diferentes ideas y enfoques, mejorando tus habilidades interpersonales y tu capacidad de pensar de forma divergente. Esto es mucho más valioso que simplemente chatear con un bot. Así que no seas tímido: ¡habla, pregunta y aprende con otros!

Sí, la IA formará parte del plan de estudios, tanto como herramienta de aprendizaje como tema en sí mismo. Incluso tendrás la oportunidad de crear tu propio software de IA. Para obtener más información sobre nuestro enfoque progresivo, consulta la documentación disponible en la intranet.

Buenas prácticas:

Estoy atascado con un nuevo concepto. Le pregunto a alguien cercano cómo lo abordó. Hablamos durante 10 minutos y, de repente, todo encaja. Lo entiendo.

Mala práctica:

Uso IA en secreto y copio código que parece correcto. Durante la evaluación por pares, no puedo explicar nada. Suspenso. Durante el examen, sin IA, me quedo atascado de nuevo. Suspenso.

Capítulo IV

Ejercicio 00: Yyyyyy... ¡ABIERTO!

	Ejercicio: 00
Yyyyyy... ¡ABIERTO!	
Directorio: ex00/	
Archivos a enviar: Makefile, main.cpp, ClapTrap.{h, hpp}, ClapTrap.cpp Prohibido: Ninguno	

Primero, ¡tienes que implementar una clase! ¡Qué original!

Se llamará ClapTrap y tendrá los siguientes atributos privados inicializados con los valores especificados entre paréntesis:

- Nombre, que se pasa como parámetro al constructor.
- Puntos de vida (10), que representan la salud del ClapTrap
- Puntos de energía (10)
- Daño de ataque (0)

Agregue las siguientes funciones miembro públicas para que ClapTrap se comporte de manera más realista:

- ataque void(const std::string& target);
- void takeDamage(unsigned int cantidad);
- void beRepaired(unsigned int cantidad);

Cuando ClapTrap ataca, hace que su objetivo pierda <daño de ataque> puntos de golpe. Cuando ClapTrap se repara, recupera <cantidad> puntos de vida. Atacar y reparar cada uno cuesta 1 punto de energía. Por supuesto, ClapTrap no puede hacer nada si no le quedan puntos de vida ni de energía. Sin embargo, dado que estos ejercicios son solo una introducción, las instancias de ClapTrap no deben interactuar directamente entre sí, y los parámetros no harán referencia a otra instancia de ClapTrap.

En todas estas funciones miembro, es necesario imprimir un mensaje que describa lo que sucede. Por ejemplo, la función `attack()` podría mostrar algo como esto (sin los corchetes angulares, claro está):

¡ClapTrap <nombre> ataca a <objetivo> y le causa <daño> puntos de daño!

Los constructores y destructores también deben mostrar un mensaje, para que sus evaluadores pares Se puede ver fácilmente que han sido llamados.

Implemente y entregue sus propias pruebas para garantizar que su código funcione como se espera.

Capítulo V

Ejercicio 01: Serena, mi amor!

	Ejercicio: 01
¡Serena, mi amor!	
Directorio: ex01/	
Archivos a enviar: Archivos del ejercicio anterior + ScavTrap.{h, hpp}, ScavTrap.cpp Prohibido: Ninguno	

Como nunca puedes tener suficientes ClapTraps, ahora crearás un robot derivado. Se llamará ScavTrap y heredará los constructores y el destructor de Clap-Trap. Sin embargo, sus constructores, destructor y ataque() mostrarán mensajes diferentes. Después de todo, los ClapTraps son conscientes de su individualidad.

Tenga en cuenta que en sus pruebas se debe mostrar un encadenamiento de construcción/destrucción adecuado. Al crear una ScavTrap, el programa comienza construyendo una ClapTrap. La destrucción ocurre en orden inverso. ¿Por qué?

ScavTrap utilizará los atributos de ClapTrap (actualizará ClapTrap según corresponda) y debe inicializarlos a:

- Nombre, que se pasa como parámetro al constructor.
- Puntos de vida (100), que representan la salud del ClapTrap
- Puntos de energía (50)
- Daño de ataque (20)

ScavTrap también tendrá su propia habilidad especial:

```
vacío guardGate();
```

Esta función miembro mostrará un mensaje indicando que ScavTrap ahora está en modo de guardián de puerta.

No olvides agregar más pruebas a tu programa.

Capítulo VI

Ejercicio 02: Trabajo repetitivo

	Ejercicio: 02
Trabajo repetitivo	
Directorio: ex02/	
Archivos a enviar: Archivos de ejercicios anteriores + FragTrap.{h, hpp}, FragTrap.cpp Prohibido:	
Ninguno	

Probablemente estés empezando a ponerte nervioso al hacer ClapTraps.

Ahora, implemente una clase FragTrap que herede de ClapTrap. Es muy similar a ScavTrap. Sin embargo, sus mensajes de construcción y destrucción deben ser diferentes. Las pruebas deben mostrar un encadenamiento correcto de construcción/destrucción. Al crear una FragTrap, el programa comienza construyendo una ClapTrap. La destrucción se realiza en orden inverso.

Lo mismo ocurre con los atributos, pero esta vez con valores diferentes:

- Nombre, que se pasa como parámetro al constructor.
- Puntos de vida (100), que representan la salud del ClapTrap
- Puntos de energía (100)
- Daño de ataque (30)

FragTrap también tiene una habilidad especial:

```
vacío highFivesGuys(vacío);
```

Esta función miembro muestra una solicitud de chocar los cinco positiva en la salida estándar.

Nuevamente, agregue más pruebas a su programa.

Capítulo VII

Ejercicio 03: ¡Ahora es raro!

	Ejercicio: 03
¡Ahora es raro!	
Directorio: ex03/	
Archivos a enviar: Archivos de ejercicios anteriores + DiamondTrap.{h, cpp}, DiamondTrap.cpp Prohibido: Ninguno	

En este ejercicio, crearás un monstruo: un ClapTrap mitad FragTrap, mitad ScavTrap. Se llamará DiamondTrap y heredará tanto de FragTrap como de ScavTrap. ¡Es muy arriesgado!

La clase DiamondTrap tendrá un atributo privado llamado name. Este atributo debe tener exactamente el mismo nombre de variable que la clase base ClapTrap (sin hacer referencia al nombre del robot).

Para ser más claro, aquí hay dos ejemplos:
 si la variable de ClapTrap es nombre, asígnele a la variable de DiamondTrap el nombre nombre.
 Si la variable de ClapTrap es _name, dale a la variable de DiamondTrap el nombre _name.

Sus atributos y funciones miembro se heredarán de sus clases principales:

- Nombre, que se pasa como parámetro a un constructor
- ClapTrap::name (parámetro del constructor + sufijo "_clap_name")
- Puntos de golpe (FragTrap)
- Puntos de energía (ScavTrap)
- Daño de ataque (FragTrap)
- ataque() (ScavTrap)

Además de las funciones especiales de ambas clases principales, DiamondTrap tendrá su propia habilidad especial:

```
vacío quienSoy();
```

Esta función miembro mostrará tanto su nombre como su nombre ClapTrap.

Por supuesto, la instancia ClapTrap de DiamondTrap se creará una vez, y solo una vez. Sí, hay un truco.

Nuevamente, agregue más pruebas a su programa.



¿Conoces los indicadores del compilador -Wshadow y -Wno-shadow?



Puedes aprobar este módulo sin completar el ejercicio 03.

Capítulo VIII

Presentación y evaluación por pares

Entregue su tarea en su repositorio de Git como de costumbre. Solo se evaluará el trabajo dentro de su repositorio durante la defensa. No dude en verificar los nombres de sus carpetas y archivos para asegurarse de que sean correctos.

Durante la evaluación, ocasionalmente se podría solicitar una breve modificación del proyecto. Esto podría implicar un cambio menor de comportamiento, la escritura o reescritura de algunas líneas de código, o una función fácil de añadir.

Si bien este paso puede no ser aplicable a todos los proyectos, debes estar preparado para ello si se menciona en las pautas de evaluación.

Este paso tiene como objetivo verificar su comprensión real de una parte específica del proyecto. La modificación se puede realizar en cualquier entorno de desarrollo que elija (por ejemplo, su configuración habitual) y debería ser factible en unos pocos minutos, a menos que se defina un período de tiempo específico como parte de la evaluación.

Por ejemplo, se le puede pedir que realice una pequeña actualización a una función o script, modifique una pantalla o ajuste una estructura de datos para almacenar nueva información, etc.

Los detalles (alcance, objetivo, etc.) se especificarán en las pautas de evaluación y pueden variar de una evaluación a otra para el mismo proyecto.